অধ্যায় ১৯

JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Performance-এর বিভিন্ন মাত্রা—সময়, সাড়াদানের ক্ষমতা, Memory, Network—আলাদা করতে।
- Big-O দিয়ে একটি Algorithm-এর বৃদ্ধির হার বিশ্লেষণ করতে, এবং কাজের ধাপ গুনে তা যাচাই করতে।
- নির্ভরযোগ্যভাবে মাপতে: Warm-up, একাধিক পুনরাবৃত্তি, Median, এবং পরিবেশের বিবরণ।
- Benchmark কীভাবে ভুল পথে নিয়ে যায় তা চিনতে।
- Main Thread-এর দীর্ঘ কাজ ছোট টুকরোয় ভাগ করে Page সচল রাখতে।
- DOM-এর হালনাগাদ একত্রে করতে এবং Layout Thrashing এড়াতে।
- Garbage Collection-এর মূল নীতি এবং সাধারণ Memory Leak চিনতে, এবং `WeakMap` ব্যবহার করতে।
- সীমিত আকারের LRU Cache তৈরি করতে।
- একটি সম্পূর্ণ **PerfLab** Benchmark ও Optimization Toolkit তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৯: DOM, `replaceChildren`।
- অধ্যায় ১০: Event Loop, Task ও Microtask।
- অধ্যায় ১২: ES Module, Dynamic `import()`।
- অধ্যায় ১৪: Memoization, Debounce ও Throttle।
- অধ্যায় ১৮: `Map` Index ও Join।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি Program সঠিক হলেই যথেষ্ট নয়; তাকে যথেষ্ট দ্রুতও হতে হয়। ব্যবহারকারী একটি Button চাপার পর দুই সেকেন্ড কিছু না ঘটলে ধরে নেন Page নষ্ট; একটি Mobile ফোনে পাঁচ MB-র JavaScript নামতে নামতে তিনি চলে যান; একটি Server-এর Memory ধীরে ধীরে বাড়তে বাড়তে এক সপ্তাহ পর বন্ধ হয়ে যায়।

কিন্তু Performance-এর কাজে সবচেয়ে বড় ঝুঁকি হলো ভুল জায়গায় সময় দেওয়া। Donald Knuth-এর বিখ্যাত সতর্কবাণী: "Premature optimization is the root of all evil"—না মেপে Optimize করা প্রায় সব সময় Code-কে জটিল করে, অথচ দ্রুত করে না। এই অধ্যায়ের মূল শিক্ষা তাই একটি পদ্ধতি: **আগে মাপুন, তারপর সবচেয়ে বড় সমস্যাটি ঠিক করুন, তারপর আবার মাপুন।**

১৯.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Memoization (অধ্যায় ১৪)	LRU Cache: সীমিত আকার, Eviction, মেয়াদ
Debounce, Throttle (অধ্যায় ১০, ১৪)	দীর্ঘ কাজ টুকরোয় ভাগ, `requestAnimationFrame`
`Map` Index দিয়ে Join (অধ্যায় ১৮)	Big-O বিশ্লেষণ, ধাপ গোনা
ReDoS-এর সময় মাপা (অধ্যায় ১৭)	নির্ভরযোগ্য Benchmark পদ্ধতি
Dynamic `import()` (অধ্যায় ১২)	Lazy Loading, Bundle-এর আকার
—	Garbage Collection, Memory Leak, `WeakMap`

১৯.২ Performance-এর মাত্রা

মাত্রা	প্রশ্ন	মাপার উপায়
Latency	একটি কাজ কত সময় নেয়?	`performance.now()`, DevTools Performance Panel
Responsiveness	ব্যবহারকারীর কাজে Page কত দ্রুত সাড়া দেয়?	দীর্ঘ কাজ (৫০ মিলিসেকেন্ডের বেশি), INP
Throughput	প্রতি সেকেন্ডে কত কাজ?	Benchmark-এ ops/sec
Memory	কত Memory, এবং তা কি বাড়তেই থাকে?	Heap Snapshot, `process.memoryUsage()`
Network	কত Byte, কতগুলো অনুরোধ?	Network Panel, Lighthouse

Browser-এ Main Thread একটি কাজে ৫০ মিলিসেকেন্ডের বেশি আটকে থাকলে তাকে **Long Task** বলা হয়, কারণ তখন Click বা লেখা চোখে পড়ার মতো দেরিতে সাড়া পায়। Google-এর **Core Web Vitals** এই অভিজ্ঞতাগুলো মাপে: LCP (মূল বিষয়বস্তু কত দ্রুত দেখা যায়), INP (ব্যবহারকারীর কাজে কত দ্রুত সাড়া), CLS (Layout কতটা অপ্রত্যাশিতভাবে সরে)।

১৯.৩ Big-O: বৃদ্ধির হার

Big-O বলে ইনপুট বাড়লে কাজ **কী হারে** বাড়ে, সেকেন্ডে কত সময় নয়। মাপার সবচেয়ে নিশ্চিত উপায়: কাজের ধাপ গোনা, যা যন্ত্রভেদে বদলায় না।

count-steps.js · JavaScript — চলবে: Browser Console ও Node.js

```
function commonNested(a, b) {
  let steps = 0;
  const result = a.filter((x) => b.some((y) => {
    steps += 1;
    return x === y;
  }));
  return { result: result.length, steps };
}

function commonWithSet(a, b) {
  let steps = b.length;
  const lookup = new Set(b);
  const result = a.filter((x) => {
    steps += 1;
    return lookup.has(x);
  });
  return { result: result.length, steps };
}

for (const n of [100, 1000, 10000]) {
  const a = Array.from({ length: n }, (_, i) => i * 2);
  const b = Array.from({ length: n }, (_, i) => i * 3);
  const nested = commonNested(a, b);
  const set = commonWithSet(a, b);
  console.log(n, nested.result === set.result, nested.steps, set.steps);
}
```

Output

```
100 true 7756 200
1000 true 777556 2000
10000 true 77775556 20000
```

ইনপুট ১০ গুণ হলে Nested পদ্ধতির ধাপ প্রায় ১০০ গুণ (**$O(n^2)$**), Set পদ্ধতির ঠিক ১০ গুণ (**$O(n)$**)। ফল দুটোরই এক (দ্বিতীয় কলাম)। ১০,০০০ উপাদানে পার্থক্য প্রায় চার হাজার গুণ—এমন পার্থক্য কোনো ছোটখাটো Optimization দিয়ে পূরণ হয় না। Algorithm-এর পছন্দই Performance-এর সবচেয়ে বড় লিভার।

Big-O	নাম	উদাহরণ	n = ১০,০০০-এ মোটামুটি ধাপ
O(1)	ধ্রুবক	`Map.get`, `Set.has`, Array-র Index	১
O(log n)	লগারিদমিক	সাজানো তালিকায় Binary Search	১৪
O(n)	রৈখিক	`includes`, `find`, একটি Loop	১০,০০০
O(n log n)		`sort`	১,৩০,০০০
$O(n^2)$	বর্গ	Loop-এর ভেতরে `includes` বা `find`	১০ কোটি

সতর্কতা: Big-O ধ্রুবক ও ছোট ইনপুট উপেক্ষা করে। ২০টি উপাদানে একটি সরল Array-তে `includes` প্রায়ই `Set` তৈরির চেয়ে দ্রুত, কারণ Set তৈরিরও খরচ আছে। Big-O বলে কোনটি **বড় হলে** টিকবে; ছোট ইনপুটে মাপুন।

১৯.৪ নির্ভরযোগ্যভাবে মাপা

একবার `performance.now()` দিয়ে মাপা সময় প্রায় অর্থহীন। JavaScript Engine (V8) প্রথমে Code ধীরে চালায়, তারপর ঘনঘন চলা অংশ অপ্টিমাইজড Machine Code-এ Compile করে (**JIT**); Garbage Collection মাঝে মাঝে সবকিছু থামায়; অন্য Program CPU ভাগ করে নেয়। একটি নির্ভরযোগ্য পরিমাপের পাঁচটি নিয়ম:

১. **Warm-up:** মাপার আগে কয়েকবার চালান, যাতে JIT-এর প্রাথমিক খরচ মাপে না ঢোকে (অধ্যায় ১৭-এর ReDoS উদাহরণ)।

২. **বহুবার, তারপর Median:** গড় একটি বিরল দীর্ঘ বিরতিতে (GC) বিকৃত হয়; Median ও p95 (৯৫% পরিমাপ যার নিচে) বেশি বিশ্বাসযোগ্য।

৩. **ফল ব্যবহার করুন:** যে হিসাবের ফল কেউ ব্যবহার করে না, Engine তা পুরো বাদ দিতে পারে (Dead Code Elimination), এবং আপনি "শূন্য সময়" মাপবেন।

৪. **একই অবস্থায় তুলনা:** একই ইনপুট, একই যন্ত্র, একই Node.js সংস্করণ; ল্যাপটপ চার্জে আছে কি না পর্যন্ত।

৫. **পরিবেশ লিখে রাখুন:** কোন CPU, কোন Node.js বা Browser, কোন OS। অন্য পরিবেশে ফল ভিন্ন হবে, এবং তা স্বাভাবিক।

stats.js · JavaScript — চলবে: Browser Console ও Node.js

```
function summarize(samples) {
  const sorted = samples.toSorted((a, b) => a - b);
  const last = sorted.length - 1;
  const at = (q) => sorted[Math.min(last, Math.floor(q * sorted.length))];
  const mean = samples.reduce((a, b) => a + b, 0) / samples.length;
  return { mean, median: at(0.5), p95: at(0.95), min: sorted[0] };
}

const samples = [10, 11, 10, 12, 11, 10, 95, 11, 10, 12];
console.log(JSON.stringify(summarize(samples)));
```

Output

```
{"mean":19.2,"median":11,"p95":95,"min":10}
```

দশটি পরিমাপের একটিতে Garbage Collection-এর বিরতি (৯৫)। গড় বলছে ১৯.২—বাস্তবের প্রায় দ্বিগুণ; Median বলছে ১১, যা স্বাভাবিক অবস্থা। p95 দেখায় সবচেয়ে খারাপ অভিজ্ঞতাগুলো কেমন—ব্যবহারকারীর দৃষ্টিতে সেটিও গুরুত্বপূর্ণ। Project-এর Benchmark এই পরিসংখ্যানগুলোই দেয়।

> **সতর্কতা:** Benchmark-এর সংখ্যা কখনো "সব সময় দ্রুত"-এর প্রমাণ নয়; প্রমাণ শুধু এই ইনপুটে, এই পরিবেশে। একটি Micro-benchmark-এ ১০% পার্থক্য প্রায়ই পরিমাপের গোলমালের মধ্যেই পড়ে। এবং Benchmark-কে Automated Test-এর Pass/Fail শর্ত বানাবেন না: যন্ত্র ব্যস্ত থাকলে Test এলোমেলোভাবে ব্যর্থ হবে।

১৯.৫ Main Thread: দীর্ঘ কাজ টুকরোয় ভাগ

অধ্যায় ১০-এ দেখেছেন, JavaScript-এর একটি দীর্ঘ Loop চলার সময় Timer, Click, পর্দা আঁকা—সব অপেক্ষা করে। সমাধান: কাজকে ছোট টুকরোয় ভাগ করে প্রতিটি টুকরোর পর Event Loop-কে সুযোগ দেওয়া। Browser-এ দেখুন:

chunks.js · JavaScript — চলবে: Browser

```
const items = Array.from({ length: 300000 }, (_, i) => i);
const work = (n) => Math.sqrt(n) * Math.sin(n);

function blocking() {
  let timerRan = false;
  setTimeout(() => {
    timerRan = true;
  }, 0);
  let total = 0;
  for (const n of items) {
    total += work(n);
  }
  return { timerRan, total };
```

```
}

async function chunked(size = 20000) {
  let timerRan = false;
  setTimeout(() => {
    timerRan = true;
  }, 0);
  let total = 0;
  let chunks = 0;
  for (let i = 0; i < items.length; i += size) {
    for (const n of items.slice(i, i + size)) {
      total += work(n);
    }
    chunks += 1;
    await new Promise((resolve) => setTimeout(resolve, 0));
  }
  return { timerRan, total, chunks };
}

const a = blocking();
console.log(`একটানা: কাজ চলাকালীন Timer চলেছে? ${a.timerRan}`);
const b = await chunked();
console.log(`টুকরোয়: Timer চলেছে? ${b.timerRan}, টুকরো ${b.chunks}`);
console.log(`একই ফল? ${Math.abs(a.total - b.total) < 1e-6}`);
```

Output

```
একটানা: কাজ চলাকালীন Timer চলেছে? false
টুকরোয়: Timer চলেছে? true, টুকরো 15
একই ফল? true
```

একটানা Loop-এর সময় একটি `setTimeout(..., 0)`-ও চলতে পারেনি, ঠিক যেমন একটি Click বা লেখা পারত না। টুকরোয় ভাগ করা সংস্করণে প্রতিটি টুকরোর পর `await` Event Loop-কে সুযোগ দিয়েছে, এবং Timer মাঝপথেই চলেছে; ফল হুবহু এক। (Listing-টি Module হিসেবে চলে, কারণ এতে Top-level `await`।) নতুন Browser-এ এ কাজের জন্য বিশেষ `scheduler.yield()` আছে; পুরোনো Browser-এর জন্য `setTimeout`-ই সর্বজনীন। সত্যিকারের ভারী হিসাব Main Thread থেকেই সরিয়ে দেওয়া যায় **Web Worker**-এ, যা আলাদা Thread-এ চলে এবং Message দিয়ে কথা বলে।

১৯.৬ DOM: হালনাগাদ একত্রে, Layout Thrashing নয়

Browser প্রতিটি DOM পরিবর্তনের পর পর্দা নতুন করে হিসাব করতে পারে: কোন Element কোথায়, কত বড় (**Layout**), তারপর আঁকা (**Paint**)। সাধারণত Browser পরিবর্তনগুলো জমিয়ে একবারে হিসাব করে। কিন্তু Code যদি একটি পরিবর্তনের পরেই একটি মাপ **পড়ে** (`offsetHeight`, `getBoundingClientRect`), Browser-কে তখনই হিসাব করতে হয়। লেখা ও পড়া পর্যায়ক্রমে চললে প্রতিটি ধাপে একটি পূর্ণ Layout—একে বলে **Layout Thrashing**।

batch-dom.js · JavaScript চলবে: Browser

```
const list = document.createElement('ul');
document.body.append(list);
const names = Array.from({ length: 500 }, (_, i) => `সারি ${i + 1}`);

// ধীর ধরন: প্রতিটি সারি যোগ, তারপর সঙ্গে সঙ্গে উচ্চতা পড়া (500টি Layout)।
for (const name of names) {
  const item = document.createElement('li');
  item.textContent = name;
  list.append(item);
  void list.offsetHeight;
}
const slowCount = list.children.length;

// দ্রুত ধরন: সব তৈরি, একবারে যোগ, শেষে একবার পড়া (1টি Layout)।
list.replaceChildren(...names.map((name) => {
  const item = document.createElement('li');
  item.textContent = name;
  return item;
}));
const height = list.offsetHeight;

console.log(slowCount, list.children.length, height > 0);
console.log(list.lastElementChild.textContent);
```

Output

```
500 500 true
সারি 500
```

দুটি পদ্ধতির ফল একই DOM; পার্থক্য Browser-এর কাজে। DevTools-এর Performance Panel-এ প্রথম পদ্ধতিতে বেগুনি "Layout" ঘটনার একটি দীর্ঘ সারি দেখা যায়; দ্বিতীয়টিতে একটি। নিয়ম: **আগে সব পড়ুন, তারপর সব লিখুন।** অধ্যায় ৯ থেকে আমরা যে `replaceChildren(...elements)` ব্যবহার করে আসছি, তা এ কারণেই ভালো অভ্যাস। ব্যবহারকারী দেখতে পান এমন Animation-এর হালনাগাদ `requestAnimationFrame`-এ রাখুন: Browser ঠিক পরের ছবি আঁকার আগে Callback চালায়, এবং লুকানো Tab-এ চালায় না।

একটি হাজারো সারির তালিকার জন্য সবচেয়ে কার্যকর Optimization হলো সারিগুলো আঁকাই না: পর্দায় যা দেখা যায় শুধু সেটুকু (**Virtualization**), অথবা Pagination।

১৯.৭ Memory এবং Garbage Collection

JavaScript-এ Memory হাতে মুক্ত করতে হয় না: **Garbage Collector (GC)** নিয়মিত খোঁজে কোন Object আর **পৌঁছানো যায় না** (Unreachable)—কোনো Variable, Closure, অন্য Object বা Global থেকে কোনো পথ নেই—এবং সেগুলো মুক্ত করে। তাই Memory Leak মানে এমন Object, যা আর দরকার নেই কিন্তু কোথাও থেকে এখনো পৌঁছানো যায়।

সাধারণ Leak	কেন মুক্ত হয় না	সমাধান
সীমাহীন Cache বা Global Array	Global সব সময় পৌঁছানো যায়	আকারের সীমা (LRU), মেয়াদ
সরানো হয়নি এমন Event Listener	Listener-এর Closure তথ্য ধরে রাখে	`removeEventListener`, বা `AbortController`-এর `signal`
চলতে থাকা `setInterval`	Callback ও তার Closure জীবিত	`clearInterval`
Page থেকে সরানো কিন্তু Variable-এ রাখা DOM	Detached Element	Reference মুছে ফেলা
বড় তথ্য ধরে রাখা Closure	Closure যতদিন, তথ্যও ততদিন	শুধু প্রয়োজনীয় অংশ রাখা

`WeakMap`-এর Key দুর্বলভাবে ধরা থাকে: Key Object অন্য কোথাও থেকে আর পৌঁছানো না গেলে Entry-টি নিজেই মুছে যায়। তাই কোনো Object-এর সঙ্গে বাড়তি তথ্য জুড়তে, Object-টিকে বাঁচিয়ে না রেখে, `WeakMap` উপযুক্ত:

`weakmap.js` · JavaScript — চলবে: Browser Console ও Node.js

```
const metadata = new WeakMap();

function track(element) {
  const info = metadata.get(element) ?? { clicks: 0 };
  info.clicks += 1;
  metadata.set(element, info);
  return info.clicks;
}

let button = { id: 'save' };
track(button);
console.log(track(button), metadata.has(button));

button = null;
console.log(metadata.has({ id: 'save' }));
console.log(typeof metadata.size, typeof metadata.keys);
```

Output

```
2 true
false
undefined undefined
```

`button = null`-এর পর পুরোনো Object আর কোথাও থেকে পৌঁছানো যায় না, তাই GC তার সঙ্গে `metadata`-র Entry-টিও মুক্ত করতে পারে (কখন করবে, তা GC-র সিদ্ধান্ত)। একটি সাধারণ `Map` হলে Map-ই Object-কে বাঁচিয়ে রাখত —Leak। এর মূল্য: `WeakMap`-এর `size` বা `keys` নেই, কারণ কোন Entry এই মুহূর্তে জীবিত তা অনিশ্চিত। নতুন Object দিয়ে খোঁজা (`{ id: 'save' }`) কাজ করে না, কারণ `WeakMap` পরিচয় (Reference) দেখে, বিষয়বস্তু নয়।

Memory মাপা: Chrome DevTools-এর Memory Panel-এ দুটি Heap Snapshot নিয়ে তুলনা করলে দেখা যায় কোন ধরনের Object বাড়ছে; Node.js-এ `process.memoryUsage().heapUsed`। একটি Leak-এর লক্ষণ: একই কাজ বারবার করলে Memory সমতলে স্থির না হয়ে সিঁড়ির মতো বাড়তেই থাকে।

১৯.৮ Cache: LRU, মেয়াদ এবং ঝুঁকি

অধ্যায় ১৪-এর Memoization-এর Cache সীমাহীন—দীর্ঘজীবী Server-এ এটি নিজেই একটি Leak। **LRU (Least Recently Used)** Cache একটি নির্দিষ্ট সংখ্যার বেশি Entry রাখে না; ভর্তি হলে সবচেয়ে বেশি দিন অব্যবহৃতটি বাদ দেয় (**Eviction**)। JavaScript-এর `Map` Entry যোগ করার ক্রম মনে রাখে, তাই LRU খুব সহজে লেখা যায়:

lru.js · JavaScript — চলবে: Node.js

```
class LruCache {
  #limit;
  #map = new Map();

  constructor(limit) {
    this.#limit = limit;
  }

  get(key) {
    if (!this.#map.has(key)) {
      return undefined;
    }
    const value = this.#map.get(key);
    this.#map.delete(key);
    this.#map.set(key, value);
    return value;
  }

  set(key, value) {
    this.#map.delete(key);
    this.#map.set(key, value);
    if (this.#map.size > this.#limit) {
      this.#map.delete(this.#map.keys().next().value);
    }
  }

  get keys() {
    return [...this.#map.keys()];
  }
}

const cache = new LruCache(3);
['a', 'b', 'c'].forEach((key) => cache.set(key, key.toUpperCase()));
cache.get('a');
```

```
cache.set('d', 'D');
console.log(cache.keys, cache.get('b'));
```

Output

```
[ 'c', 'a', 'd' ] undefined
```

`get` একটি Entry-কে মুছে আবার যোগ করে, ফলে সেটি "সাম্প্রতিকতম" প্রান্তে চলে যায়। ভর্তি হলে `Map`-এর প্রথম Key —সবচেয়ে পুরোনো ব্যবহৃত—বাদ। `a` সম্প্রতি পড়া হয়েছিল বলে বেঁচে গেছে, `b` বাদ পড়েছে।

Cache-এর ঝুঁকি: **বাসি তথ্য** (মূল তথ্য বদলালেও Cache পুরোনোটি দেয়—তাই মেয়াদ বা TTL), **Memory** (তাই আকারের সীমা), এবং **ভুল Key** (দুটি ভিন্ন অনুরোধ একই Key পেলে একজনের তথ্য অন্যজন পায়—ব্যবহারকারীভিত্তিক তথ্যে এটি নিরাপত্তা-ঝুঁকিও)। প্রবাদটি: Computer Science-এর দুটি কঠিন সমস্যার একটি হলো Cache কখন অবৈধ করতে হবে তা জানা।

১৯.৯ Network: Byte-ই সবচেয়ে ধীর

Browser-এ প্রায়ই সবচেয়ে বড় খরচ JavaScript চালানো নয়, আনা। প্রতিটি KB ধীর Mobile Network-এ সময় নেয়, এবং আসার পর Parse ও Compile করতেও CPU লাগে।

- **কম পাঠান:** অব্যবহৃত Code বাদ (Bundler-এর **Tree Shaking**, যা ES Module-এর স্থির `import`-এর কারণে সম্ভব—অধ্যায় ১২), এবং একটি ছোট কাজের জন্য বিশাল Library এড়ানো।
- **পরে পাঠান (Lazy Loading):** যা প্রথম পর্দায় লাগে না, তা প্রয়োজনের মুহূর্তে Dynamic `import()` দিয়ে (অধ্যায় ১২-এর সমাধান ১১)। ছবির জন্য `<img loading="lazy">`।
- **সংকুচিত করে পাঠান:** Server-এর gzip বা Brotli Compression; ছবির উপযুক্ত আকার ও Format (WebP, AVIF)।
- **একবার পাঠান:** Cache Header (`Cache-Control`), এবং File-এর নামে বিষয়বস্তুর Hash (`app.3f9a1c.js`), যাতে বদলালে নতুন নাম, না বদলালে দীর্ঘ মেয়াদি Cache।
- **মাপুন:** DevTools-এর Network Panel-এ "Slow 4G" Throttling, এবং Lighthouse Report।

১৯.১০ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
না মেপে Optimize	জটিল Code, লাভ নেই	আগে Profile
একবারের পরিমাপে সিদ্ধান্ত	গোলমালকে পার্থক্য ভাবা	Warm-up, বহুবার, Median
গড় দিয়ে বিচার	একটি GC-বিরতি গড় বিকৃত করে	Median ও p95
Benchmark-এর ফল ব্যবহার না করা	Engine হিসাব বাদ দেয়	ফল জমিয়ে রাখুন

ভুল	কেন সমস্যা	সমাধান
Loop-এর ভেতরে `includes` / `find`	$O(n^2)$	`Set` / `Map` Index
ছোট ইনপুটে Big-O-র কথা ভেবে জটিলতা	ধ্রুবক খরচ বেশি	মাপুন
লেখা-পড়া পর্যায়ক্রমে DOM-এ	Layout Thrashing	আগে পড়া, পরে লেখা
একটানা ভারী Loop Main Thread-এ	Page জমে যায়	টুকরো, বা Web Worker
সীমাহীন Cache	Memory Leak	LRU, মেয়াদ
Listener ও Interval সরানো হয় না	Memory Leak	পরিষ্কারের ব্যবস্থা

১৯.১১ Debugging নির্দেশিকা

১. **সমস্যা নির্দিষ্ট করুন:** "ধীর" নয়; "১০,০০০ সারির Report তৈরিতে ৪ সেকেন্ড, লক্ষ্য ৫০০ মিলিসেকেন্ড"।

২. **Profile করুন:** Chrome DevTools-এর Performance Panel-এ Record করে কাজটি করুন; Flame Chart-এ সবচেয়ে চওড়া Function-গুলো খুঁজুন। Node.js-এ `node --cpu-prof script.mjs` একটি `.cpuprofile` File তৈরি করে, যা DevTools-এ খোলা যায়।

৩. **সবচেয়ে বড় খরচটি আগে:** একটি Function যদি মোট সময়ের ৮০% নেয়, বাকিগুলো দ্বিগুণ দ্রুত করেও সামান্য লাভ।

৪. **একটি পরিবর্তন, তারপর আবার মাপুন,** একই ইনপুটে; ফল বা আচরণ বদলায়নি তা Test দিয়ে নিশ্চিত করুন।

৫. **Memory:** দুটি Heap Snapshot-এর তুলনা; Detached DOM Element ও সীমাহীন Collection খুঁজুন।

৬. **Long Task:** Performance Panel-এ লাল কোণযুক্ত ৫০ মিলিসেকেন্ডের বেশি কাজগুলো।

১৯.১২ পেশাদার অভ্যাস

- **আগে সঠিক, তারপর পরিষ্কার, তারপর দ্রুত**—এবং দ্রুত করার আগে মাপা।
- **Algorithm আগে, Micro-optimization পরে।**
- **প্রতিটি Optimization-এর সঙ্গে একটি Correctness Test,** যা প্রমাণ করে ফল বদলায়নি।
- **Benchmark-এর ফলের সঙ্গে পরিবেশ ও পদ্ধতি লিখে রাখুন;** একক সংখ্যা নয়।
- **Main Thread হালকা:** ৫০ মিলিসেকেন্ডের বেশি কাজ ভাগ করুন।
- **প্রতিটি Cache-এর আকার ও মেয়াদের সীমা।**
- **যা চালু করেন, তা বন্ধের ব্যবস্থা:** Listener, Interval, Subscription।
- **Performance-এর বাজেট:** যেমন "প্রথম পর্দার JavaScript ১৭০ KB-র কম"; প্রতিটি পরিবর্তনে যাচাই।

‘‘

১৯.১৩ Project: PerfLab

PerfLab একটি ছোট Performance Toolkit (Node.js): একটি নির্ভরযোগ্য Benchmark Harness (Warm-up, পুনরাবৃত্তি, Median, p95, পরিবেশের বিবরণ), একই কাজের ধীর ও দ্রুত Algorithm-জোড়া, একটি LRU Cache (আকার, মেয়াদ, Hit-Miss পরিসংখ্যান), এবং দীর্ঘ কাজ টুকরোয় ভাগের একটি সাহায্যকারী। এর সবচেয়ে গুরুত্বপূর্ণ অংশ Benchmark নয়, Test: প্রতিটি দ্রুত Algorithm তার ধীর সংস্করণের সঙ্গে ফলে হুবহু মেলে কি না, তা এলোমেলো কিন্তু পুনরুৎপাদনযোগ্য ইনপুটে যাচাই।

Project-এর চাহিদা

- Harness: Warm-up, নির্দিষ্ট সংখ্যক পরিমাপ, Median, p95, সর্বনিম্ন, ops/sec; সময়ের উৎস Inject করা যায় (Test-এর জন্য)।
- Algorithm-জোড়া: পুনরাবৃত্তি বাদ (Nested বনাম `Set`), সাধারণ উপাদান (Nested বনাম `Set`), সাজানো তালিকায় খোঁজা (Linear বনাম Binary)।
- Correctness: প্রতিটি জোড়া একই ইনপুটে একই ফল, Seed-যুক্ত এলোমেলো ইনপুটে, যাতে ব্যর্থ হলে আবার একই ইনপুট পাওয়া যায়।
- LRU Cache: সীমা, মেয়াদ (TTL, ঘড়ি Inject করা), Hit/Miss গণনা।
- টুকরোয় কাজ: সাহায্যকারী প্রতিটি টুকরোর পর একটি Inject করা "বিরতি" Function ডাকে।
- Benchmark Report: Markdown, পরিবেশসহ; Benchmark কখনো Test-এর শর্ত নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
perf-lab/
├── src/
│   ├── bench.mjs          Benchmark Harness ও পরিসংখ্যান
│   ├── algorithms.mjs     ধীর ও দ্রুত Algorithm-জোড়া
│   ├── lru.mjs            LRU Cache (TTL, পরিসংখ্যান)
│   ├── chunk.mjs          টুকরোয় কাজ
│   └── random.mjs         Seed-যুক্ত এলোমেলো সংখ্যা
├── run-bench.mjs          Benchmark চালিয়ে Report লেখে
├── README.md
└── tests/
    ├── algorithms.test.mjs
    ├── lru.test.mjs
    └── bench-chunk.test.mjs
```

src/random.mjs ও src/algorithms.mjs

random.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// Seed-যুক্ত ছোট PRNG (mulberry32): একই Seed-এ সব সময় একই ক্রম।
export function createRandom(seed) {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let t = state;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export const randomInts = (random, count, max) =>
  Array.from({ length: count }, () => Math.floor(random() * max));
```

`Math.random()`-এর Seed দেওয়া যায় না, তাই একটি ব্যর্থ Test-এর ইনপুট আর ফিরে পাওয়া যায় না। ছোট এই Generator (mulberry32 নামে পরিচিত) একটি সংখ্যা (Seed) থেকে সব সময় একই "এলোমেলো" ক্রম দেয়। বিটের কারসাজিগুলো (`>>>`, `Math.imul`) এখানে বোঝার দরকার নেই; শুধু জানুন এটি নিরাপত্তার কাজের উপযুক্ত নয় (তার জন্য `crypto`, অধ্যায় ২০)।

algorithms.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// প্রতিটি কাজের দুটি সংস্করণ: সরল (ধীর) এবং দ্রুত। ফল হুবহু এক হতে হবে।

export const uniqueNested = (items) =>
  items.filter((item, index) => items.indexOf(item) === index);

export const uniqueSet = (items) => [...new Set(items)];

export const commonNested = (a, b) => a.filter((x) => b.includes(x));

export function commonSet(a, b) {
  const lookup = new Set(b);
  return a.filter((x) => lookup.has(x));
}

export const indexOfLinear = (sorted, target) => sorted.indexOf(target);

// সাজানো তালিকায় প্রথম মিলের Index, বা -1। O(log n)।
export function indexOfBinary(sorted, target) {
  let low = 0;
  let high = sorted.length - 1;
  let found = -1;
  while (low <= high) {
    const middle = (low + high) >>> 1;
```

```
    if (sorted[middle] < target) {
      low = middle + 1;
    } else {
      if (sorted[middle] === target) {
        found = middle;
      }
      high = middle - 1;
    }
  }
  return found;
}
```

- **`uniqueNested`** প্রতিটি উপাদানের জন্য পুরো তালিকায় `indexOf`: $O(n^2)$। **`uniqueSet`** O(n), এবং প্রথম দেখার ক্রম রাখে—তাই দুটির ফল হুবহু এক।
- **`indexOfBinary`** প্রতিবার খোঁজার পরিসর অর্ধেক করে। মিলে গেলেও বাঁ দিকে খোঁজা চালিয়ে যায়, যাতে পুনরাবৃত্ত মানে **প্রথম** Index ফেরত দেয়—ঠিক `indexOf`-এর মতো। এই সূক্ষ্মতা বাদ পড়লে দ্রুত সংস্করণ "প্রায়" সঠিক হতো; Correctness Test এমন ভুলই ধরে।
- **`(low + high) >>> 1`** দুটি পূর্ণসংখ্যার গড়ের পূর্ণ অংশ।

src/bench.mjs

bench.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export function summarize(samples) {
  const sorted = samples.toSorted((a, b) => a - b);
  const last = sorted.length - 1;
  const at = (q) => sorted[Math.min(last, Math.floor(q * sorted.length))];
  return { median: at(0.5), p95: at(0.95), min: sorted[0], runs: samples.length };
}

// fn বহুবার চালিয়ে সময় মাপে। now Inject করা যায় (Test-এ নিয়ন্ত্রিত ঘড়ি)।
export function bench(fn, options = {}) {
  const { warmup = 5, runs = 30, now = () => performance.now() } = options;
  let sink;
  for (let i = 0; i < warmup; i += 1) {
    sink = fn();
  }
  const samples = [];
  for (let i = 0; i < runs; i += 1) {
    const start = now();
    sink = fn();
    samples.push(now() - start);
  }
  const stats = summarize(samples);
  const opsPerSec = stats.median > 0 ? 1000 / stats.median : Infinity;
  return { ...stats, opsPerSec, sinkType: typeof sink };
}
```

```
export function environment() {
  return {
    node: process.version,
    platform: `${process.platform} ${process.arch}`,
    date: new Date().toISOString().slice(0, 10),
  };
}
```

`sink` প্রতিটি ফল ধরে রাখে, এবং শেষে তার ধরন ফলের সঙ্গে ফেরত যায়; ফলটি "ব্যবহৃত", তাই Engine হিসাব বাদ দিতে পারে না (Dead Code Elimination প্রতিরোধ)। `now` Inject করা যায়: Test-এ একটি কৃত্রিম ঘড়ি দিয়ে Median ও p95-এর হিসাব নিশ্চিতভাবে যাচাই করা হয়, আসল সময়ের ওপর নির্ভর না করে।

src/lru.mjs ও src/chunk.mjs

lru.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export class LruCache {
  #limit;
  #ttl;
  #now;
  #map = new Map();
  stats = { hits: 0, misses: 0, evictions: 0 };

  constructor({ limit, ttlMs = Infinity, now = () => Date.now() }) {
    if (!Number.isInteger(limit) || limit < 1) {
      throw new RangeError('limit একটি ধনাত্মক পূর্ণসংখ্যা হতে হবে');
    }
    this.#limit = limit;
    this.#ttl = ttlMs;
    this.#now = now;
  }

  get(key) {
    const entry = this.#map.get(key);
    if (!entry || this.#now() - entry.at >= this.#ttl) {
      this.#map.delete(key);
      this.stats.misses += 1;
      return undefined;
    }
    this.#map.delete(key);
    this.#map.set(key, entry);
    this.stats.hits += 1;
    return entry.value;
  }

  set(key, value) {
    this.#map.delete(key);
```

```
    this.#map.set(key, { value, at: this.#now() });
    if (this.#map.size > this.#limit) {
      this.#map.delete(this.#map.keys().next().value);
      this.stats.evictions += 1;
    }
  }

  get size() {
    return this.#map.size;
  }
}
```

chunk.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const nextTask = () => new Promise((resolve) => setTimeout(resolve, 0));

// items-কে size-এর টুকরোয় process করে; প্রতিটি টুকরোর পর yieldFn।
export async function processInChunks(items, fn, options = {}) {
  const { size = 1000, yieldFn = nextTask } = options;
  if (!Number.isInteger(size) || size < 1) {
    throw new RangeError('size একটি ধনাত্মক পূর্ণসংখ্যা হতে হবে');
  }
  const results = [];
  for (let i = 0; i < items.length; i += size) {
    for (const item of items.slice(i, i + size)) {
      results.push(fn(item));
    }
    if (i + size < items.length) {
      await yieldFn();
    }
  }
  return results;
}
```

LRU-তে নতুন দুটি জিনিস: প্রতিটি Entry-র সময় (`at`), যা দিয়ে মেয়াদোত্তীর্ণ Entry পড়ার সময়ই বাদ পড়ে; এবং `stats`, যাতে বোঝা যায় Cache আসলে কাজে লাগছে কি না (Hit-এর হার কম হলে Cache শুধু Memory খরচ)। `processInChunks` শেষ টুকরোর পরে আর বিরতি নেয় না, এবং `yieldFn` Inject করা যায়—Browser-এ `scheduler.yield`, Test-এ একটি গণনাকারী Function।

run-bench.mjs

run-bench.mjs · JavaScript — চলবে: Node.js

```
import { writeFileSync, mkdirSync } from 'node:fs';
import { bench, environment } from './src/bench.mjs';
import { createRandom, randomInts } from './src/random.mjs';
import * as algo from './src/algorithms.mjs';
```

```
const random = createRandom(2026);
const a = randomInts(random, 5000, 10000);
const b = randomInts(random, 5000, 10000);
const sorted = a.toSorted((x, y) => x - y);
const targets = randomInts(random, 200, 10000);
const searchAll = (find) => targets.map((t) => find(sorted, t));

const cases = [
  ['unique: nested', () => algo.uniqueNested(a)],
  ['unique: Set', () => algo.uniqueSet(a)],
  ['common: nested', () => algo.commonNested(a, b)],
  ['common: Set', () => algo.commonSet(a, b)],
  ['search x200: linear', () => searchAll(algo.indexOfLinear)],
  ['search x200: binary', () => searchAll(algo.indexOfBinary)],
];

const env = environment();
const rows = cases.map(([name, fn]) => {
  const result = bench(fn, { warmup: 5, runs: 25 });
  const { median, p95, runs } = result;
  return `| ${name} | ${median.toFixed(3)} | ${p95.toFixed(3)} | ${runs} |`;
});

const report = [
  '# PerfLab Benchmark',
  '',
  `Node.js ${env.node}, ${env.platform}, ${env.date}.`,
  'n = 5000; warm-up 5, runs 25.',
  '',
  '| Case | median ms | p95 ms | runs |',
  '| --- | --- | --- | --- |',
  ...rows,
].join('\n');

mkdirSync('results', { recursive: true });
writeFileSync('results/benchmark-report.md', `${report}\n`);
console.log(report);
```

এই Script ইচ্ছা করে Automated Test-এর অংশ নয়: সময়ের সংখ্যা যন্ত্র ও মুহূর্তভেদে বদলায়। এই বইয়ের লেখার সময় একটি যন্ত্রে (Intel Core i5-6600, 8 Core, ৮ GB RAM, Windows 10) চালানো আসল Report, Companion Code-এর `results/benchmark-report.md`-এ সংরক্ষিত:

পরিবেশ: Node.js v22.16.0, win32 x64, 2026-10-15. n = 5000; warm-up 5, runs 25.

Case	median (ms)	p95 (ms)	পরিমাপ
unique: nested	13.363	33.531	25
unique: Set	0.254	1.689	25

Case	median (ms)	p95 (ms)	পরিমাপ
common: nested	44.422	108.101	25
common: Set	0.425	1.182	25
search x200: linear	0.854	2.779	25
search x200: binary	0.023	0.173	25

একই যন্ত্রে ঠিক আগের চালানোয় "common: nested"-এর Median ছিল 33.674 ms, এবারে 44.422 ms—কোনো Code না বদলেই প্রায় ৩০% পার্থক্য। একবারের পরিমাপ কেন যথেষ্ট নয়, এটিই তার জীবন্ত প্রমাণ। অনুপাতও কিছুটা বদলেছে, কিন্তু তার মাত্রা দুবারই এক: Set সংস্করণ প্রায় ৫০ থেকে ১০০ গুণ দ্রুত। সংখ্যাগুলো পড়ার নিয়ম: নিজের যন্ত্রে আপনার সংখ্যা আলাদা হবে; গুরুত্বপূর্ণ হলো **অনুপাত** এবং ইনপুট বাড়ালে সেই অনুপাত কীভাবে বদলায়। Nested বনাম Set-এর পার্থক্য Big-O-র ভবিষ্যদ্বাণীর সঙ্গে মেলে কি না দেখুন; n দ্বিগুণ করে আবার চালালে Nested-এর সময় প্রায় চারগুণ, Set-এর প্রায় দ্বিগুণ হওয়ার কথা।

Code-এর ধাপে ধাপে ব্যাখ্যা

- **একই Seed (২০২৬)** প্রতিবার একই ইনপুট দেয়, তাই দুটি চালানোর তুলনা ন্যায্য।
- **প্রতিটি Case একটি Function,** যা ফল ফেরত দেয়; `bench` সেই ফল ধরে রাখে।
- **Warm-up ৫, পরিমাপ ২৫, Median ও p95**; Report-এ পরিবেশ ও পদ্ধতি লেখা।
- **Report একটি File-এ,** তারিখসহ; পরের Optimization-এর পর একই Script চালিয়ে তুলনা করা যায়।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node --test
node run-bench.mjs
```

Companion Source Code-এর ২০টি Test: প্রতিটি Algorithm-জোড়ার ফল ১০০টি Seed-যুক্ত এলোমেলো ইনপুটে হুবহু মেলে কি না (Binary Search-এর পুনরাবৃত্ত মান ও সীমার ক্ষেত্রসহ); LRU-র Eviction ক্রম, মেয়াদ (কৃত্রিম ঘড়ি) ও পরিসংখ্যান; Harness-এর Median ও p95 (কৃত্রিম ঘড়ি); এবং টুকরোয় কাজের ফল, ক্রম ও বিরতির সংখ্যা। কোনো Test সময়ের ওপর নির্ভর করে না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
দ্রুত সংস্করণের ফল মাঝে মাঝে আলাদা	সীমার ক্ষেত্র (পুনরাবৃত্ত মান, ফাঁকা তালিকা)	Seed দিয়ে ব্যর্থ ইনপুট আবার চালান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Benchmark-এর সংখ্যা প্রতিবার আলাদা	স্বাভাবিক গোলমাল	Median, বেশি পরিমাপ, শান্ত যন্ত্র
একটি Case "শূন্য" সময়	ফল ব্যবহৃত হয়নি	`sink`
Cache-এ Hit প্রায় নেই	Key প্রতিবার আলাদা, বা সীমা ছোট	`stats` দেখুন
টুকরোয় ভাগের পরও Page জমে	টুকরো বড়	`size` কমান

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Harness পূর্ণাঙ্গ Benchmark Library-র (যেমন Tinybench বা Benchmark.js) মতো পরিসংখ্যানগত তাৎপর্য যাচাই করে না; শুধু Node.js-এ চলে, Browser-এর Rendering মাপে না। ফলাফল এক যন্ত্রের, এক মুহূর্তের।

১৯.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `[3, 1, 2]`-এ `includes` এবং `Set.has`—দুটির Big-O কত? কোনটি ১০ উপাদানে ভালো, কেন বলা কঠিন?

২. পাঁচটি পরিমাপ `[12, 13, 12, 80, 12]`-এর গড় ও Median বের করুন; কোনটি বিশ্বাসযোগ্য?

৩. Nested Loop-এ দুটি তালিকার সাধারণ উপাদান খোঁজার ধাপ গুনুন (n = ১০০); তারপর Set দিয়ে।

৪. একটি `setInterval`-এর Leak সারান, যাতে Component সরানোর সময় Interval বন্ধ হয়।

৫. `WeakMap` ও `Map`-এর একটি পার্থক্য উদাহরণসহ লিখুন।

মধ্যম স্তর

৬. দুটি পদ্ধতিতে একটি তালিকা থেকে সবচেয়ে বেশি আসা মান বের করুন (Nested ও `Map`), এবং Seed-যুক্ত ইনপুটে দেখান ফল এক।

৭. PerfLab-এর LRU দিয়ে একটি ধীর Function Memoize করুন, এবং ১০টি অনুরোধে Hit-এর সংখ্যা দেখান।

৮. `processInChunks`-কে একটি গণনাকারী `yieldFn` দিয়ে চালিয়ে দেখান ১০,০০০ উপাদানে ১,০০০-এর টুকরোয় কয়টি বিরতি হয়।

৯. একটি Event Listener-কে `AbortController`-এর `signal` দিয়ে যোগ করুন, এবং `abort()` দিয়ে সরান (Browser)।

চ্যালেঞ্জ স্তর

১০. সাজানো দুটি তালিকার সাধারণ উপাদান O(n + m)-এ বের করুন (দুটি Pointer), এবং Set পদ্ধতির সঙ্গে ফল মেলান।

১১. PerfLab-এর Harness দিয়ে `uniqueNested` ও `uniqueSet` n = ১০০০ ও ২০০০-এ মাপুন; Median-এর অনুপাত Big-O-র সঙ্গে মেলে কি না বিচার করুন (নিজের যন্ত্রে)।

১২. LRU-তে একটি `peek(key)` যোগ করুন, যা মান দেখায় কিন্তু "সাম্প্রতিক" ক্রম বদলায় না।

১৯.১৫ অনুশীলনীর সমাধান

সমাধান ১ — `includes` O(n), `Set.has` O(1)। কিন্তু ১০টি উপাদানে Set তৈরির খরচ (প্রতিটি উপাদান Hash করা) একবারের খোঁজার চেয়ে বেশি হতে পারে; ছোট n-এ ধ্রুবক খরচই নির্ণায়ক। একই তালিকায় বহুবার খুঁজলে Set একবার তৈরি করা লাভজনক। ছোট ইনপুটে উত্তর দেয় পরিমাপ, Big-O নয়।

সমাধান ২

mean-median.js · JavaScript — চলবে: Browser Console ও Node.js

```
const samples = [12, 13, 12, 80, 12];
const mean = samples.reduce((a, b) => a + b, 0) / samples.length;
const median = samples.toSorted((a, b) => a - b)[Math.floor(samples.length / 2)];

console.log(mean, median);
```

Output

```
25.8 12
```

একটি ৮০-র বিরতি গড়কে দ্বিগুণের বেশি করেছে; Median স্বাভাবিক অবস্থা দেখায়।

সমাধান ৩

count-common.js · JavaScript — চলবে: Browser Console ও Node.js

```
const a = Array.from({ length: 100 }, (_, i) => i);
const b = Array.from({ length: 100 }, (_, i) => i + 50);

let nestedSteps = 0;
for (const x of a) {
  for (const y of b) {
    nestedSteps += 1;
    if (x === y) {
      break;
    }
  }
}
```

```
const lookup = new Set(b);
let setSteps = b.length;
for (const x of a) {
  setSteps += 1;
  lookup.has(x);
}
console.log(nestedSteps, setSteps);
```

Output

```
6275 200
```

সমাধান ৪

interval-cleanup.js · JavaScript — চলবে: Browser Console ও Node.js

```
function startClock(onTick) {
  const id = setInterval(onTick, 50);
  return () => clearInterval(id);
}

let ticks = 0;
const stop = startClock(() => {
  ticks += 1;
});

setTimeout(() => {
  stop();
  const seen = ticks;
  setTimeout(() => console.log(`বন্ধের পর নতুন টিক: ${ticks - seen}`), 150);
}, 120);
```

Output

```
বন্ধের পর নতুন টিক: 0
```

যে Function কিছু চালু করে, সে বন্ধের Function ফেরত দেয়—React-এর `useEffect`-এর Cleanup এই নীতিতেই।

সমাধান ৫ — `Map` Key-কে শক্তভাবে ধরে রাখে: Key Object আর কোথাও না থাকলেও Map-ই তাকে বাঁচিয়ে রাখে। `WeakMap` দুর্বলভাবে ধরে: Key আর পৌঁছানো না গেলে Entry মুছে যেতে পারে। তাই একটি DOM Element-এর সঙ্গে বাড়তি তথ্য রাখতে `WeakMap`: Element Page থেকে সরে গেলে তথ্যও মুক্ত। মূল্য: `WeakMap`-এর Key শুধু Object, এবং `size` বা Loop নেই (অনুচ্ছেদ ১৯.৭-এর উদাহরণ)।

সমাধান ৬

most-frequent.mjs · JavaScript — চলবে: Node.js

```
import { createRandom, randomInts } from '../../project/perf-lab/src/random.mjs';

const mostNested = (items) => items.reduce((best, item) => {
  const count = items.filter((x) => x === item).length;
  return count > best.count || (count === best.count && item < best.value)
    ? { value: item, count }
    : best;
}, { value: undefined, count: 0 });

function mostMap(items) {
  const counts = new Map();
  let best = { value: undefined, count: 0 };
  for (const item of items) {
    const count = (counts.get(item) ?? 0) + 1;
    counts.set(item, count);
    if (count > best.count || (count === best.count && item < best.value)) {
      best = { value: item, count };
    }
  }
  return best;
}

const random = createRandom(7);
let same = true;
for (let round = 0; round < 50; round += 1) {
  const items = randomInts(random, 200, 30);
  same &&= JSON.stringify(mostNested(items)) === JSON.stringify(mostMap(items));
}
console.log(same);
```

Output

```
true
```

সমান সংখ্যায় সবচেয়ে ছোট মান—এই নিয়ম দুটিতে একই থাকায় ফল নিশ্চিত; নিয়মটি না থাকলে দুটি সঠিক Algorithm-ও ভিন্ন "সঠিক" উত্তর দিত, এবং তুলনা অর্থহীন হতো।

সমাধান ৭

lru-memo.mjs · JavaScript — চলবে: Node.js

```
import { LruCache } from '../../project/perf-lab/src/lru.mjs';
```

```
const cache = new LruCache({ limit: 3 });
const slowSquare = (n) => n * n;

function cachedSquare(n) {
  const hit = cache.get(n);
  if (hit !== undefined) {
    return hit;
  }
  const value = slowSquare(n);
  cache.set(n, value);
  return value;
}

[1, 2, 1, 3, 1, 4, 2, 1, 5, 1].forEach(cachedSquare);
console.log(cache.stats, cache.size);
```

Output

```
{ hits: 4, misses: 6, evictions: 3 } 3
```

`1` বারবার ব্যবহৃত বলে কখনো বাদ পড়েনি; সীমা ৩ হওয়ায় Memory কখনো তিনটির বেশি নয়।

সমাধান ৮

count-yields.mjs · JavaScript — চলবে: Node.js

```
import { processInChunks } from '../../project/perf-lab/src/chunk.mjs';

let pauses = 0;
const items = Array.from({ length: 10000 }, (_, i) => i);
const results = await processInChunks(items, (n) => n * 2, {
  size: 1000,
  yieldFn: async () => {
    pauses += 1;
  },
});

console.log(results.length, results.at(-1), pauses);
```

Output

```
10000 19998 9
```

দশটি টুকরো, কিন্তু বিরতি নয়টি: শেষ টুকরোর পর বিরতির প্রয়োজন নেই।

সমাধান ৯

abort-listener.js · JavaScript — চলবে: Browser

```
const button = document.createElement('button');
button.id = 'go';
button.textContent = 'চাপুন';
document.body.append(button);

const controller = new AbortController();
let clicks = 0;

button.addEventListener('click', () => {
  clicks += 1;
  console.log(`Click ${clicks}`);
  controller.abort();
}, { signal: controller.signal });
```

দুবার চাপলে:

Output

```
Click 1
```

প্রথম Click-এর পর `abort()` Listener সরিয়ে দিয়েছে; দ্বিতীয় Click-এ কিছুই চলেনি। একই `signal` দিয়ে অনেক Listener যোগ করলে একটি `abort()` সবগুলো একসঙ্গে সরায়—একটি Component বন্ধের সময় সব পরিষ্কারের সহজ উপায়।

সমাধান ১০

two-pointers.mjs · JavaScript — চলবে: Node.js

```
import { createRandom, randomInts } from '../../project/perf-lab/src/random.mjs';
import { commonSet } from '../../project/perf-lab/src/algorithms.mjs';

function commonSorted(a, b) {
  const result = [];
  let i = 0;
  let j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] < b[j]) {
      i += 1;
    } else if (a[i] > b[j]) {
      j += 1;
    } else {
      result.push(a[i]);
      i += 1;
```

```
      j += 1;
    }
  }
  return result;
}

const random = createRandom(42);
const unique = (n) =>
  [...new Set(randomInts(random, n, 500))].toSorted((x, y) => x - y);
const a = unique(300);
const b = unique(300);

const fast = JSON.stringify(commonSorted(a, b));
const same = fast === JSON.stringify(commonSet(a, b));
console.log(same);
```

Output

```
true
```

দুটি Pointer প্রতি ধাপে অন্তত একটি এগোয়, তাই মোট ধাপ সর্বোচ্চ n + m, এবং কোনো বাড়তি Memory লাগে না। শর্ত: দুটি তালিকা সাজানো এবং পুনরাবৃত্তিহীন।

সমাধান ১১ — `bench(() => uniqueNested(a1000))` ও `bench(() => uniqueNested(a2000))`-এর Median-এর অনুপাত প্রায় ৪ এবং `uniqueSet`-এর প্রায় ২ হওয়ার কথা ($O(n^2)$ ও $O(n)$)। বাস্তবে Cache, JIT ও ছোট n-এর কারণে অনুপাত কিছুটা আলাদা হবে; কয়েকবার চালিয়ে দেখুন অনুপাত স্থির কি না। একবারের সংখ্যা থেকে সিদ্ধান্ত নেবেন না, এবং ফলের সঙ্গে নিজের যন্ত্রের বিবরণ লিখে রাখুন।

সমাধান ১২

peek.mjs · JavaScript — চলবে: Node.js

```
import { LruCache } from '../../project/perf-lab/src/lru.mjs';

class PeekableCache extends LruCache {
  #shadow = new Map();

  set(key, value) {
    super.set(key, value);
    this.#shadow.set(key, value);
  }

  peek(key) {
    return this.#shadow.get(key);
  }
}
```

```
const cache = new PeekableCache({ limit: 2 });
cache.set('a', 1);
cache.set('b', 2);
console.log(cache.peek('a'));
cache.set('c', 3);
console.log(cache.get('a'), cache.get('b'));
```

Output

```
1
undefined 2
```

`peek` `a`-কে "সাম্প্রতিক" করেনি, তাই `c` যোগে `a`-ই বাদ পড়েছে। লক্ষ করুন, এই সমাধান একটি দুর্বলতা দেখায়: Child Class Parent-এর Private `#map`-এ পৌঁছাতে পারে না, তাই আলাদা `#shadow` রাখতে হয়েছে, যা বাদ পড়া Entry মুছে না দিলে নিজেই Leak হবে। ভালো নকশা: `peek` LRU-র নিজের Class-এ যোগ করা (অধ্যায় ১৩-এর Composition বনাম Inheritance)।

১৯.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Loop-এর ভেতরে `includes` দিয়ে দুটি তালিকার সাধারণ উপাদান খোঁজার Big-O?

ক) O(n)

খ) $O(n^2)$

গ) O(log n)

ঘ) O(1)

প্রশ্ন ২. Benchmark-এ Warm-up কেন?

ক) JIT Compile-এর প্রাথমিক খরচ মাপ থেকে বাদ দিতে

খ) CPU গরম করতে

গ) ফল বদলাতে

ঘ) Memory কমাতে

প্রশ্ন ৩. একটি GC-বিরতিসহ পরিমাপে কোনটি স্বাভাবিক অবস্থা ভালো দেখায়?

ক) গড়

খ) সর্বোচ্চ

গ) Median

ঘ) যোগফল

প্রশ্ন ৪. Benchmark-এর ফল ব্যবহার না করলে কী হতে পারে?

ক) কিছুই না

খ) Program ভেঙে যায়

গ) সময় দ্বিগুণ হয়

ঘ) Engine হিসাব বাদ দিয়ে অবাস্তব কম সময় দেখাতে পারে

প্রশ্ন ৫. Browser-এ "Long Task" কী?

ক) ৫০ মিলিসেকেন্ডের বেশি Main Thread আটকে রাখা কাজ

খ) Network-এর দীর্ঘ অনুরোধ

গ) বড় File

ঘ) অনেক লাইনের Function

প্রশ্ন ৬. Layout Thrashing-এর কারণ?

ক) অনেক CSS

খ) DOM-এ লেখা ও মাপ পড়া পর্যায়ক্রমে

গ) বড় ছবি

ঘ) `textContent`

প্রশ্ন ৭. Garbage Collector কোন Object মুক্ত করে?

ক) পুরোনোগুলো

খ) বড়গুলো

গ) যেগুলো আর কোথাও থেকে পৌঁছানো যায় না

ঘ) `null` বসানো প্রতিটি Variable-এর মান, তৎক্ষণাৎ

প্রশ্ন ৮. একটি DOM Element-এর সঙ্গে বাড়তি তথ্য রাখতে, Element-কে বাঁচিয়ে না রেখে, কোনটি?

ক) `Map`

খ) Array

গ) Global Object

ঘ) `WeakMap`

প্রশ্ন ৯. LRU Cache ভর্তি হলে কোন Entry বাদ দেয়?

ক) সবচেয়ে দীর্ঘ সময় অব্যবহৃত

খ) সবচেয়ে নতুন

গ) সবচেয়ে বড়

ঘ) এলোমেলো

> **প্রশ্ন ১০.** Benchmark-কে Automated Test-এর Pass/Fail শর্ত কেন বানানো উচিত নয়?
>
> ক) Benchmark ধীর
>
> খ) সময় যন্ত্রের অবস্থাভেদে বদলায়; Test এলোমেলোভাবে ব্যর্থ হবে
>
> গ) Test Runner সময় মাপতে পারে না
>
> ঘ) Benchmark-এর ফল সব সময় সঠিক

১৯.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	প্রতিটি উপাদানে পুরো তালিকা খোঁজা।
২	ক	প্রথম চালানো ধীর, অপ্টিমাইজড Code পরে আসে।
৩	গ	একটি বিরল দীর্ঘ মান Median-কে প্রায় বদলায় না।
৪	ঘ	Dead Code Elimination।
৫	ক	তখন ব্যবহারকারীর কাজ দেরিতে সাড়া পায়।
৬	খ	প্রতিটি পড়া Browser-কে তখনই Layout হিসাব করতে বাধ্য করে।
৭	গ	Reachability-ই মাপকাঠি; কখন মুক্ত করবে, তা GC ঠিক করে।
৮	ঘ	দুর্বল Reference; Element সরে গেলে তথ্যও মুক্ত হতে পারে।
৯	ক	Least Recently Used।
১০	খ	Correctness Test নিশ্চিত হওয়া উচিত; সময় নয়।

১৯.১৮ অধ্যায়ের সারসংক্ষেপ

- Performance মানে সময়, সাড়া, Throughput, Memory ও Network; আগে মাপুন, তারপর সবচেয়ে বড় সমস্যা।
- Big-O বৃদ্ধির হার; ধাপ গুনে যাচাই; Loop-এর ভেতরে খোঁজা $O(n^2)$, Set/Map Index $O(n)$।
- নির্ভরযোগ্য পরিমাপ: Warm-up, বহুবার, Median ও p95, ফল ব্যবহার, পরিবেশ লেখা।
- Main Thread-এ ৫০ মিলিসেকেন্ডের বেশি কাজ টুকরোয় ভাগ, বা Web Worker।
- DOM: আগে পড়া, পরে লেখা; একত্রে হালনাগাদ; বড় তালিকায় Virtualization।
- GC Unreachable Object মুক্ত করে; Leak মানে অপ্রয়োজনীয় কিন্তু পৌঁছানো যায় এমন Object; `WeakMap`।
- Cache-এর আকার (LRU) ও মেয়াদের সীমা; Hit-এর হার মাপুন।

- Network: কম, পরে, সংকুচিত, একবার পাঠান।
- প্রতিটি Optimization-এর সঙ্গে Correctness Test; Benchmark Test-এর শর্ত নয়।

পরের অধ্যায়ে JavaScript Security: Threat Modeling, XSS, Injection, Authentication ও Authorization, এবং নিরাপদ Code লেখার অভ্যাস।

১৯.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Long Task	৫০ মিলিসেকেন্ডের বেশি Main Thread আটকানো কাজ।
Big-O	ইনপুটের সঙ্গে কাজের বৃদ্ধির হার।
JIT	চলার সময় ঘনঘন Code-কে অপ্টিমাইজড Machine Code-এ রূপান্তর।
Median / p95	মাঝের মান / ৯৫% মান যার নিচে।
Dead Code Elimination	অব্যবহৃত ফলের হিসাব বাদ দেওয়া।
Layout Thrashing	লেখা-পড়া পর্যায়ক্রমে বারবার Layout।
Garbage Collection	পৌঁছানো যায় না এমন Object-এর Memory মুক্ত করা।
Memory Leak	অপ্রয়োজনীয় Object জীবিত থেকে যাওয়া।
LRU Cache	সবচেয়ে দীর্ঘ অব্যবহৃত Entry বাদ দেওয়া সীমিত Cache।
Lazy Loading	প্রয়োজনের মুহূর্তে আনা।

১৯.২০ আরও পড়ুন

- **MDN — Performance.now():** `https://developer.mozilla.org/en-US/docs/Web/API/Performance/now` — উচ্চ-নির্ভুল সময় মাপা।
- **web.dev — Optimize long tasks:** `https://web.dev/articles/optimize-long-tasks` — Main Thread-এ কাজ ভাগ করা।
- **MDN — Memory management:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_management` — Garbage Collection ও Reachability।
- **Chrome DevTools — Analyze runtime performance:** `https://developer.chrome.com/docs/devtools/performance` — Performance Panel।
- **web.dev — Web Vitals:** `https://web.dev/articles/vitals` — LCP, INP, CLS।